

AI-Assisted Power BI Development

Copilot, Agents, and Beyond





Marc Lelijveld

Technical Evangelist | Solution Architect
Macaw Netherlands



 [linkedin.com/in/MarcLelijveld](https://www.linkedin.com/in/MarcLelijveld)

 Data-Marc.com

FAVORITE STUFF:



Using AI is just...



Why good prompts are crucial



AI output quality depends heavily on the input you give



Clear prompts reduce hallucinations and irrelevant answers



Saves time → fewer retries and corrections



Essential for reliable business use of AI

Risks of poor prompts

Wrong conclusions due to assumptions



Inconsistent AI answers

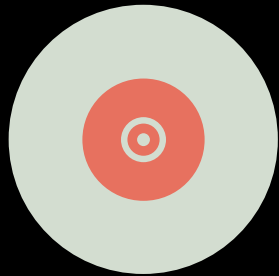


Loss of trust in AI tools



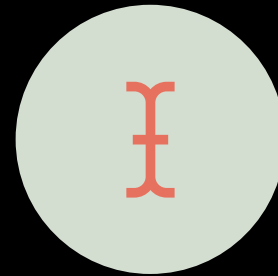
Possible governance / data exposure risks

What makes a prompt **strong**?



Context

background, goal,
audience



Clarity

specific, unambiguous
wording



Constraints

format, tone, length,
scope



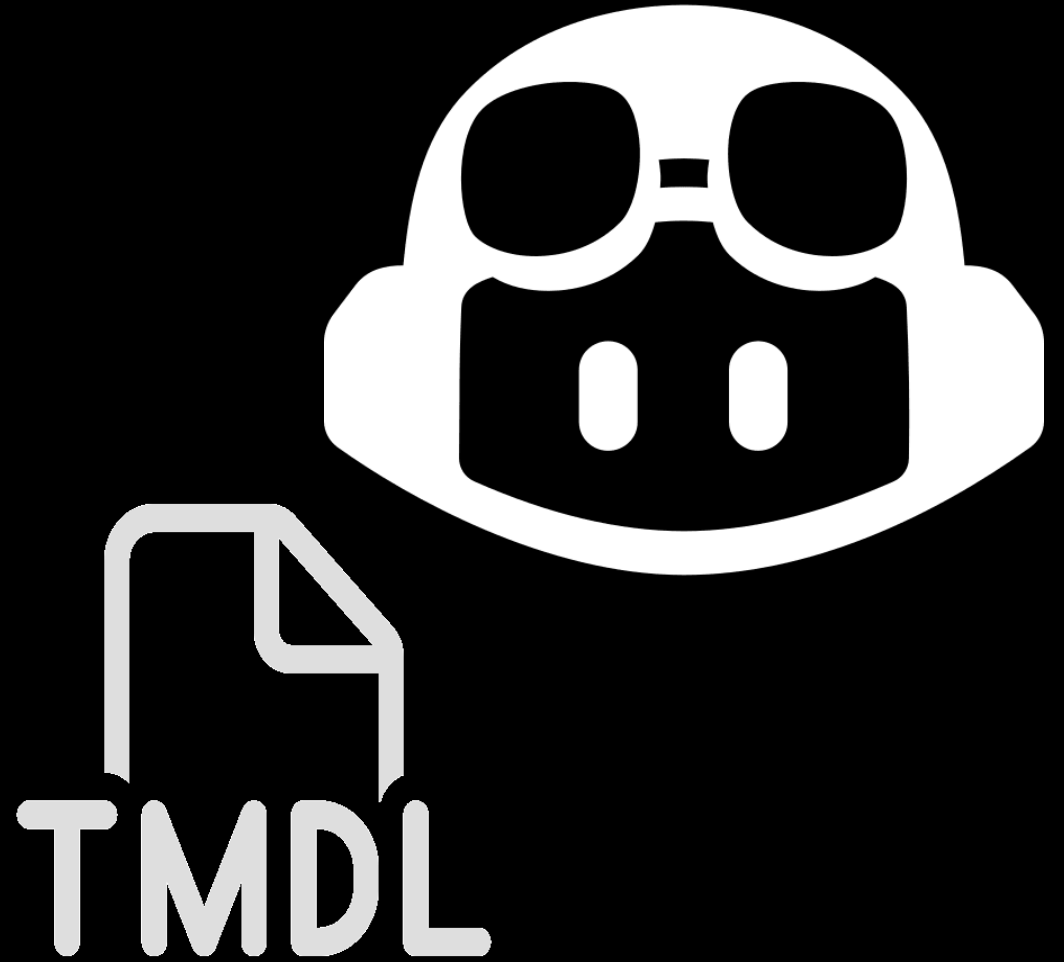
Examples

show what “good”
looks like

So... we have too many topics, you choose!

- Prep data for AI – optimizing Semantic Models to improve AI outcomes
 - Chat with your data through Copilot
 - Governance and control
- AI assisted Power BI development
 - Dataflows gen2 & copilot
 - Smart narrative visual
 - Copilot in DAX Query View (DQV)
- VS Code + Extensions (like Power BI Modeling MCP)
 - TMDL changes using GitHub Copilot + Skills
 - Auto-generate ERD diagram using drawio + GitHub Copilot

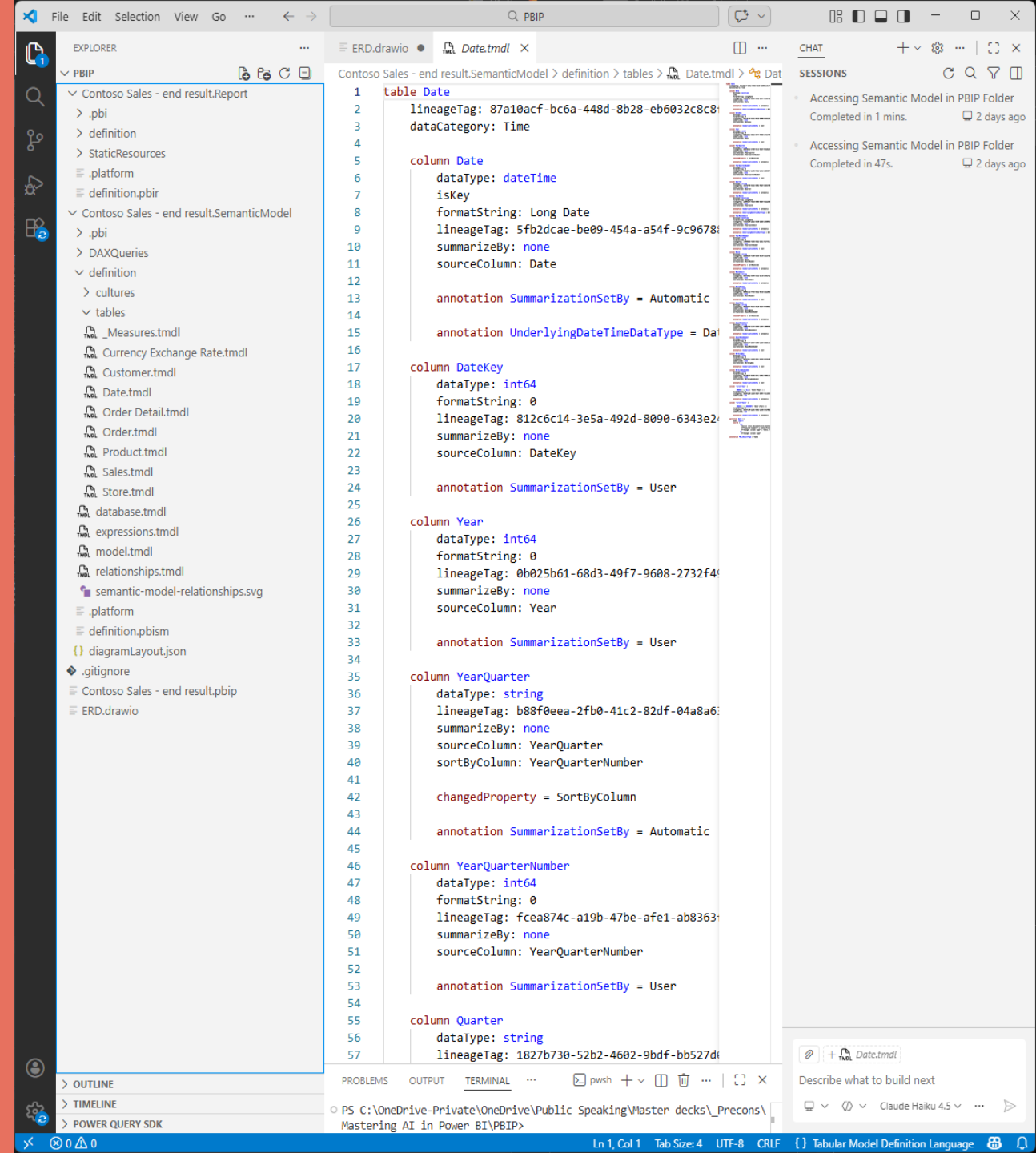
GitHub Copilot & TMDL





Visual Studio Code (VS Code) is a lightweight, powerful, and extensible source code editor

- Developed by Microsoft
- Free and open-source
- Cross-platform (Windows, macOS, Linux)
- Supports hundreds of programming languages
- Git integration (also relevant for PBI)



VS Code Extensions

VS Code Extensions add new capabilities to Visual Studio Code



Power BI Modeling MCP Server

Brings Power BI semantic modeling capabilities to your agents.



Power Query

Language service for the Power Query / M formula language.



GitHub Copilot Chat

AI peer programming tool that helps you write code faster and smarter.



TMDL

Language support for the Tabular Model Definition Language (TMDL).



What is it

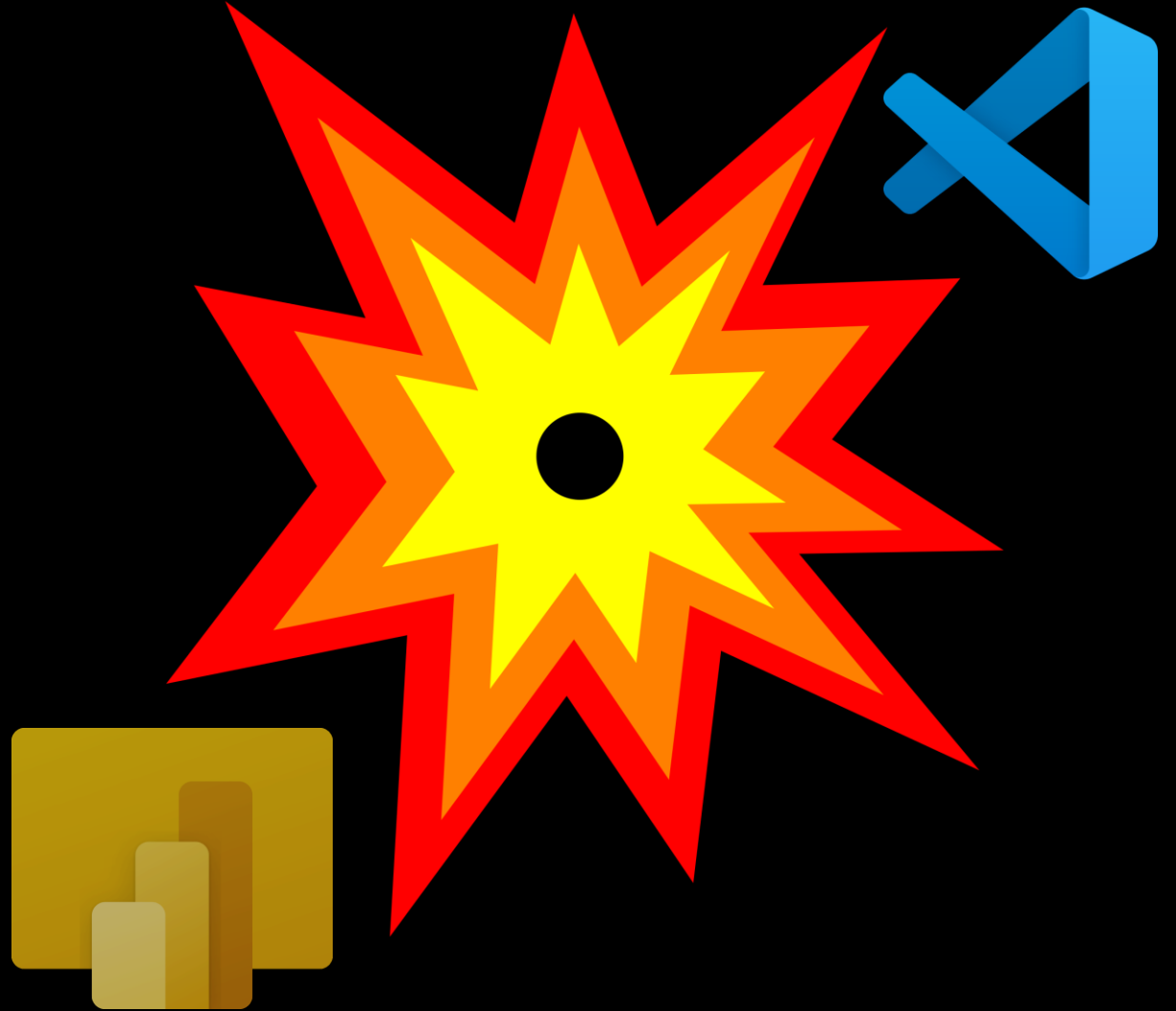
- Built by GitHub in collaboration with OpenAI
- Powered by large language models trained on code and natural language
- Integrated into editors like Visual Studio Code, Visual Studio, and GitHub.com
- Provides real-time code suggestions as you type

What it can do

- Autocomplete lines or entire functions
- Turn comments into working code
- Explain existing code
- Generate tests and documentation
- Refactor and optimize code

The clash!

Can we build a visual data
model diagram in VS Code?



Case 1: Filling the gap in working from VS Code



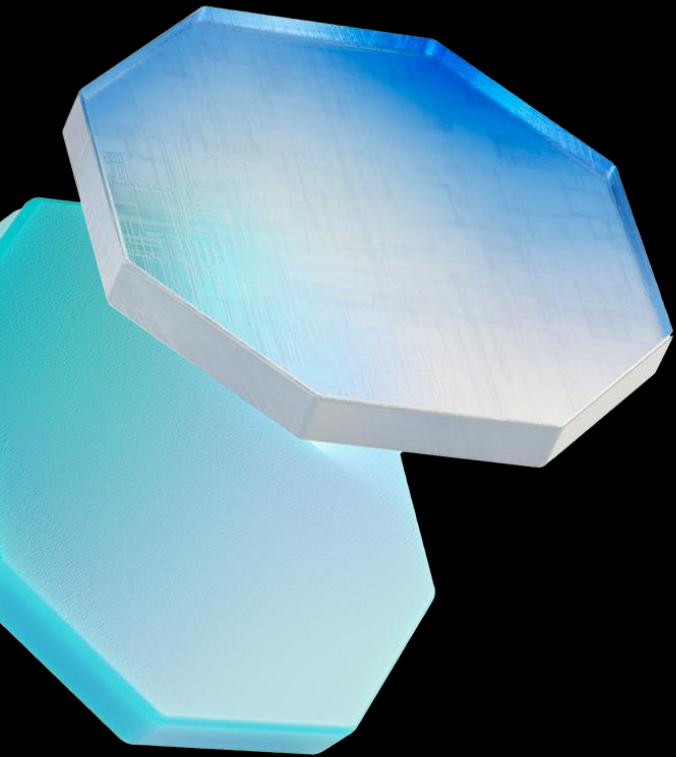
Power BI Desktop (GUI-first)

- Built-in Model View shows tables & relationships graphically. Easily see how tables connect and flow.
- Clicking a table shows its relationships and cardinality lines in a diagram.
- Data modelers can explore and validate schema visually.
- Suitable for non-dev users and quick exploration.



VS Code (Code-first)

- Working with semantic models in code means you don't see a visual ERD by default — just text and metadata files. This makes understanding the overall structure harder without extra steps.
- You're mainly editing *.tmdl files and code without seeing how entities relate visually — relationships are implicit in metadata, not visual.
- Code gives precision and version control but lacks a built-in visual layer — GREAT for automation / CI/CD but harder to see the big picture.



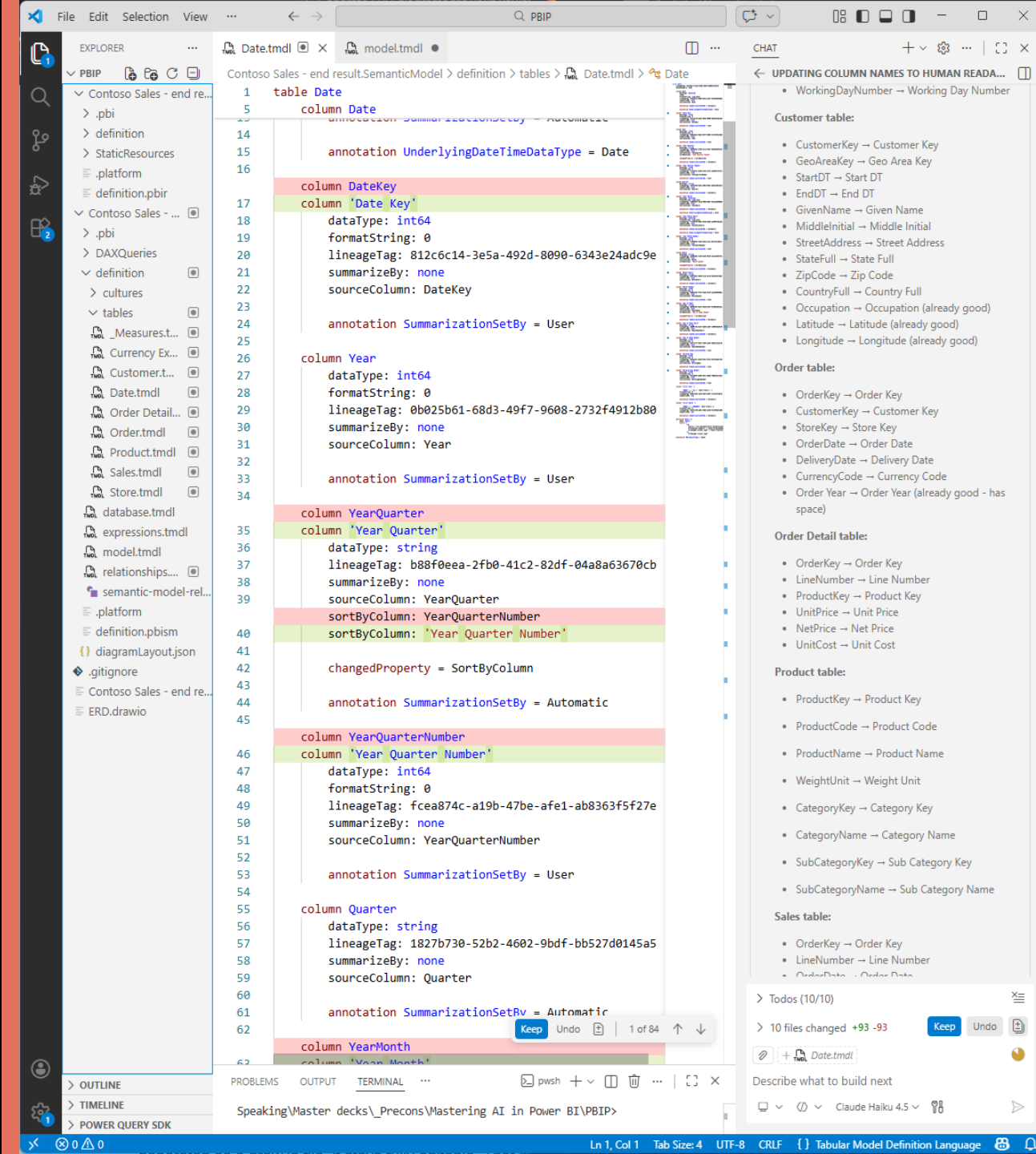
Demo

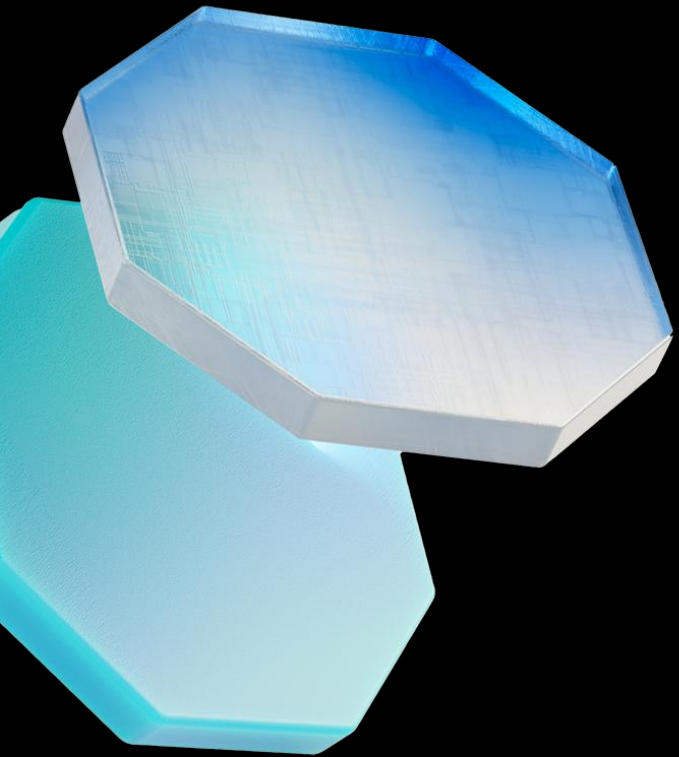
*Create ERD using GitHub
Copilot in VS Code*

Case 2: Bulk column renaming

Meeting best practices by bulk-renaming columns to human readable format

- camelCase / PascalCase renaming across all tables → affecting TMDL files only
- Breaking changes for the visuals → fixing the visual.json files

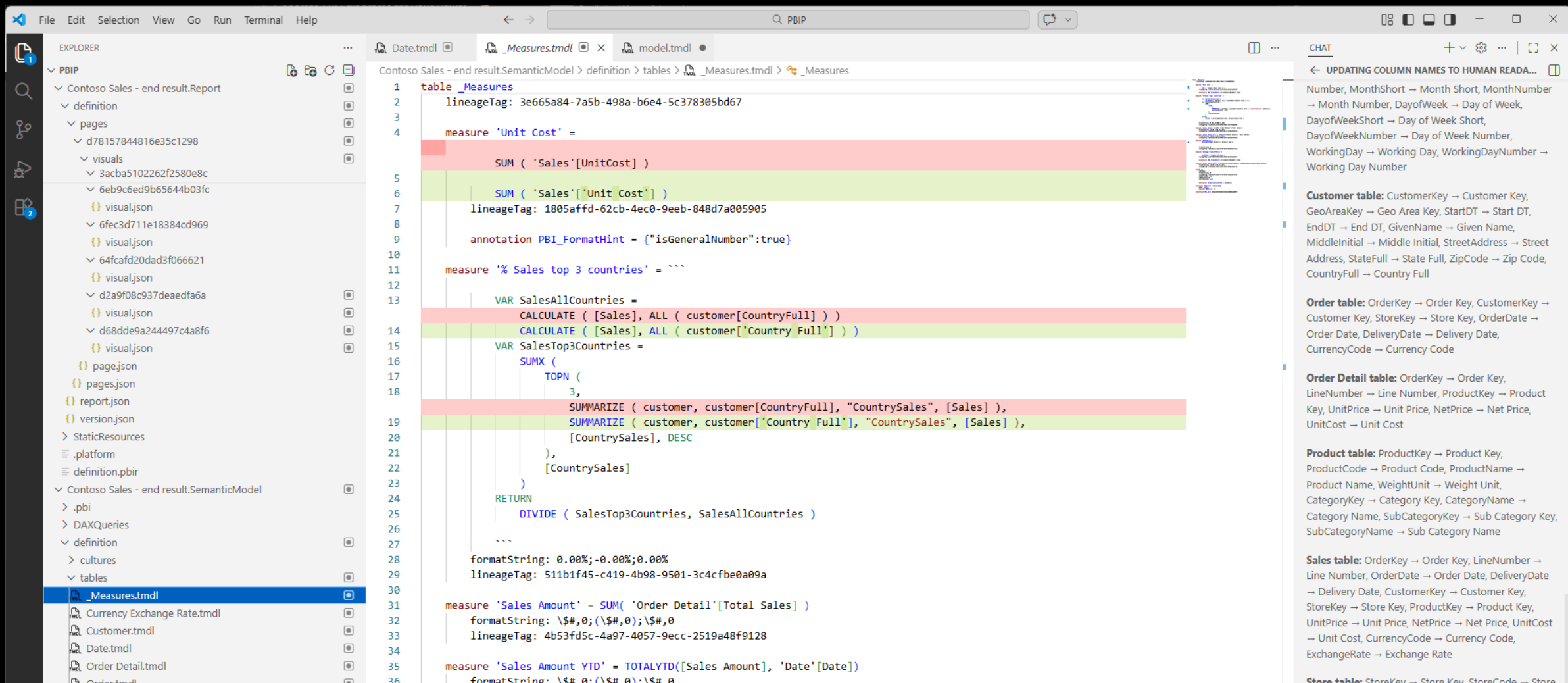




Demo

*Use VS Code + GitHub Copilot
for bulk renaming*

Even the measures are fixed!

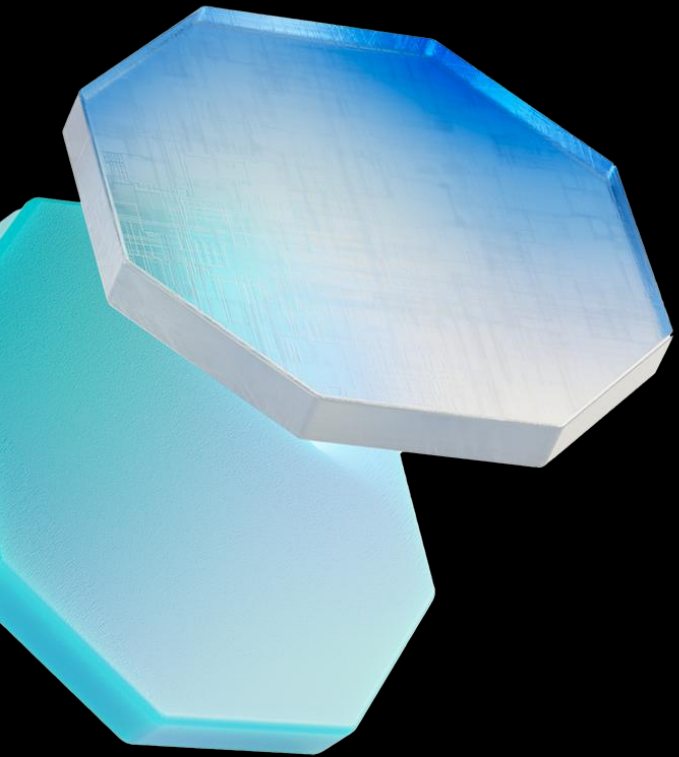


Case 2: Bulk column renaming

Cool, that worked!

Now, in the folder Contoso Sales - end result.Report, you will find a definition folder. In the definition folder you will find many visual.json files. Each of them references the old column naming, fix this by adjusting to the just renamed columns.

The screenshot displays the Visual Studio Code interface with the Tabular Model Definition Language (TMDL) editor open. The Explorer pane on the left shows the project structure, including folders like 'Contoso Sales - end result', 'definition', and 'tables'. The main editor displays the TMDL code for the 'Date' table, including column definitions for 'DateKey', 'Year', 'YearQuarter', 'YearQuarterNumber', and 'Quarter'. The right-hand side shows a 'CHAT' pane with a list of column renaming suggestions, such as 'WorkingDayNumber -> Working Day Number' and 'CustomerKey -> Customer Key'. The bottom status bar indicates the current file is 'Date.tmdl' and the language is 'Tabular Model Definition Language'.



Demo

Fixing visual references

Visual.json files corrections

Note that the measure references remain unchanged

The screenshot shows the Visual Studio Code interface. The Explorer pane on the left displays a project structure with folders like 'PBIP', 'Contoso Sales - end result.Report', and 'definition'. The 'visual.json' file is selected. The main editor shows the JSON content of 'visual.json', which includes fields like 'position', 'visual', 'query', 'queryState', 'Category', 'projections', 'field', 'Column', 'Expression', 'SourceRef', 'Entity', 'Property', 'queryRef', 'nativeQueryRef', and 'active'. The JSON is partially highlighted in pink and green. The Chat pane on the right shows a conversation with an AI assistant. The chat messages include: 'Cool, that works! Now in the folder Contoso Sales - end result.Report, you will find a definition folder. In the definition folder you will find many visual.json files. Each of them references the old column naming, fix this by adjusting to the just renamed columns.', 'I'll update all the visual.json files to use the new column names. Let me first explore the structure to understand what needs to be changed.', 'Reviewed 10 files', 'Now let me search for all column references that need updating in the visual.json files:', 'Searched for regex patterns in JSON files', 'Now I'll update the visual.json files to use the new column names:', 'Updated visual.json', 'Perfect! All the visual.json files have been updated successfully. The changes made:', and a list of updates: 'Updated 2 visual.json files (in the Product Category Name slicers and bar chart)', 'Changed "CategoryName" -> "Category Name"', and 'Updated all references: "Property", "queryRef", and "nativeQueryRef" fields'. The bottom right corner of the chat pane shows the name 'elijahveld'.

```
4    "position": {
10   },
11   "visual": {
12     "visualType": "clusteredBarChart",
13     "query": {
14       "queryState": {
15         "Category": {
16           "projections": [
17             {
18               "field": {
19                 "Column": {
20                   "Expression": {
21                     "SourceRef": {
22                       "Entity": "Product"
23                     }
24                   },
25                   "Property": "CategoryName"
26                 },
27                 "Property": "Category Name"
28               },
29               "queryRef": "Product.CategoryName",
30               "nativeQueryRef": "CategoryName",
31               "queryRef": "Product.Category Name",
32               "nativeQueryRef": "Category Name",
33               "active": true
34             }
35           ]
36         },
37         "Y": {
38           "projections": [
39             {
40               "field": {
41                 "Measure": {
42                   "Expression": {
43                     "SourceRef": {
44                       "Entity": "_Measures"
45                     }
46                   },
47                   "Property": "Sales Amount"
48                 }
49             }
50           ]
51         }
52       }
53     }
54   }
55 }
```

CHAT

← UPDATING COLUMN NAMES TO HUMAN READA...

Cool, that works! Now in the folder Contoso Sales - end result.Report, you will find a definition folder. In the definition folder you will find many visual.json files. Each of them references the old column naming, fix this by adjusting to the just renamed columns.

✓ Explored directory structure and planned updates for visual.json files

I'll update all the visual.json files to use the new column names. Let me first explore the structure to understand what needs to be changed.

✓ Reviewed 10 files

Now let me search for all column references that need updating in the visual.json files:

✓ Searched for regex patterns in JSON files

Now I'll update the visual.json files to use the new column names:

✓ Updated visual.json

Perfect! All the visual.json files have been updated successfully. The changes made:

- Updated 2 visual.json files (in the Product Category Name slicers and bar chart)
- Changed "CategoryName" -> "Category Name"
- Updated all references: "Property", "queryRef", and "nativeQueryRef" fields

All visuals now reference the new human-readable column names with spaces.

elijahveld

AI might make mistakes!

Review the changes before committing!

Column references should not be withing quotes '...'

```
1 Store Type = if(Store['Description'] = "Online Store", "Online", "Physical store")
```

! Column "Description" in table 'Store' cannot be found or may not be used in this expression.

The downside of a code editor which does not automatically adjust names in references.

Though, an easy fix, which costs less time than manually changing all column names by hand.

Power BI MCP



Improving Agentic Coding:

Agent skills, Instructions, Custom Agents, & Prompt files

Agent skills *(noun)*

Reusable capabilities you can add to GitHub Copilot agents to extend what they can do. Agent skills package tools, prompts, and logic into focused abilities (for example, interacting with an API, running a workflow, or applying domain-specific rules), allowing agents to perform structured, repeatable tasks beyond simple code generation.

Instructions *(noun)*

Custom guidance that shapes how GitHub Copilot behaves in your project. Instructions define coding standards, architectural patterns, naming conventions, or domain context so Copilot produces responses aligned with your team's way of working, instead of generic suggestions.

Custom agents *(noun)*

Purpose-built Copilot agents tailored to a specific workflow or responsibility. Custom agents combine instructions, skills, and context to act with a defined role — such as reviewer, refactoring assistant, or domain expert — enabling more focused, goal-driven collaboration inside your development environment.

Prompt files *(noun)*

Reusable prompt templates you can invoke on demand (e.g., via slash commands) to perform common development tasks consistently. Prompt files let you encode complex prompts once and reuse them across chats, reducing repetitive prompting and helping produce predictable results.

MCP *(abbr.)*

A standardized way for Copilot and other AI tools to securely discover and access external context, tools, and systems. MCP defines how models understand what capabilities are available and what actions they are allowed to perform, enabling agents and skills to operate consistently across repositories, services, and platforms.

Language models

Language Models					
Type to search...					
Name		Context Size		Capabilities	Multiplier
▼	Copilot				
	Claude Haiku 4.5	↓ 128K	↑ 16K	Tools Vision	0.33x
	Claude Opus 4.1	↓ 68K	↑ 16K	Vision	10x
	Claude Opus 4.5 (Preview)	↓ 128K	↑ 16K	Tools Vision	3x
	Claude Sonnet 4	↓ 128K	↑ 16K	Tools Vision	1x
	Claude Sonnet 4.5	↓ 128K	↑ 16K	Tools Vision	1x
	Gemini 2.5 Pro	↓ 109K	↑ 64K	Tools Vision	1x
	Gemini 3 Pro (Preview)	↓ 109K	↑ 64K	Tools Vision	1x
	GPT-4.1	↓ 111K	↑ 16K	Tools Vision	0x
	GPT-4o	↓ 124K	↑ 4K	Tools Vision	0x
🔍	GPT-5	↓ 128K	↑ 128K	Tools Vision	1x
	GPT-5 mini	↓ 128K	↑ 64K	Tools Vision	0x

Do you care about the microchip design inside RAM chips?
Or do you just care which ones makes your PC fastest?

Agent skills

Open standard (agentskills.io)

3

```
my-skill/
├── SKILL.md           # Required: instructions + metadata
├── scripts/           # Optional: executable code
├── references/        # Optional: documentation
└── assets/            # Optional: templates, resources
```

Very specific format

Use agents to create them!
(github.com/anthropics/skills)

SKILL.md

1

```
---
name: pdf-processing
description: Extract text and tables from PDF files, fill forms, merge documents.
---
```

```
# PDF Processing
```

```
## When to use this skill
```

```
Use this skill when the user needs to work with PDF files...
```

```
## How to extract text
```

```
1. Use pdfplumber for text extraction...
```

```
## How to fill forms
```

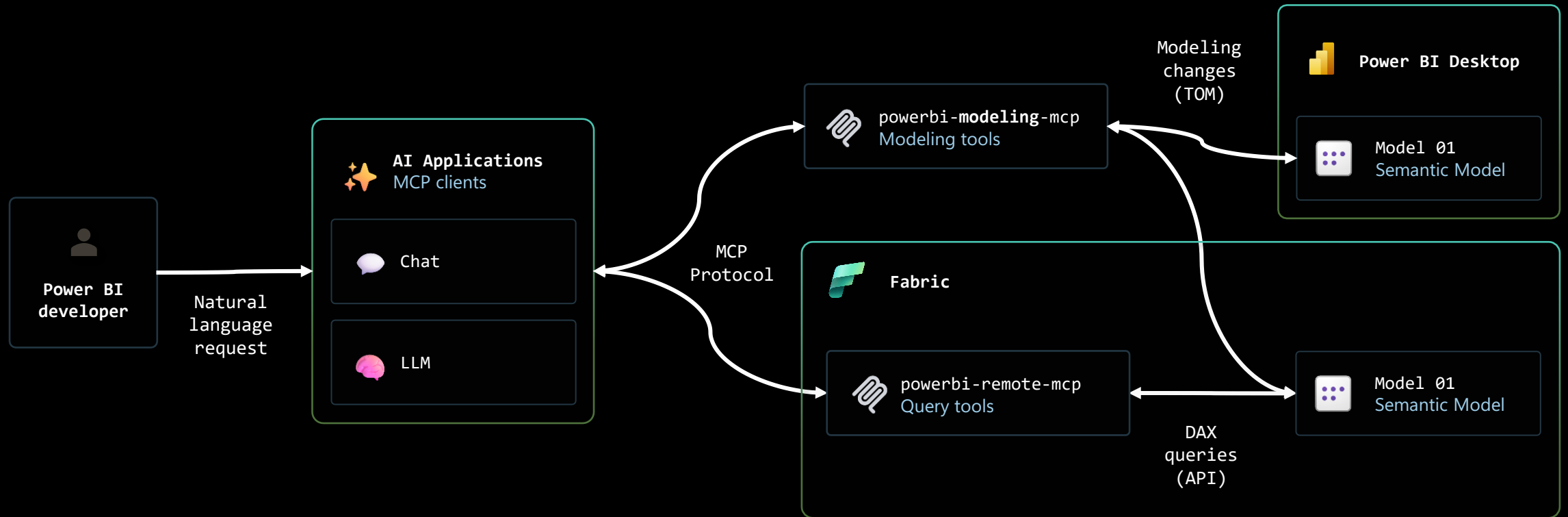
```
...
```

2

Progressive disclosure !

Power BI & MCP

Model Context Protocol is an open protocol that standardizes how AI Agents communicate with Applications or Platforms.



Modeling MCP Server

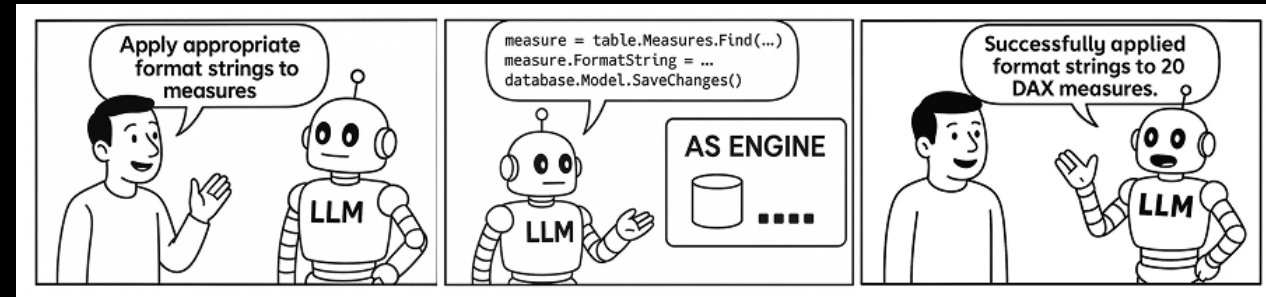
Allows AI tools to interact with Power BI's semantic layer

Agentic workflows: AI can understand, query and modify semantic models using natural language

10x Efficiency: Easily apply bulk changes turning hours of work into seconds

Best practices: review and apply best-practices

Local MCP Server: distributed as a VS Code extension but you can also manually install in any other MCP clients



Visual Studio | Marketplace

Visual Studio Code > AI > Power BI Modeling MCP Server



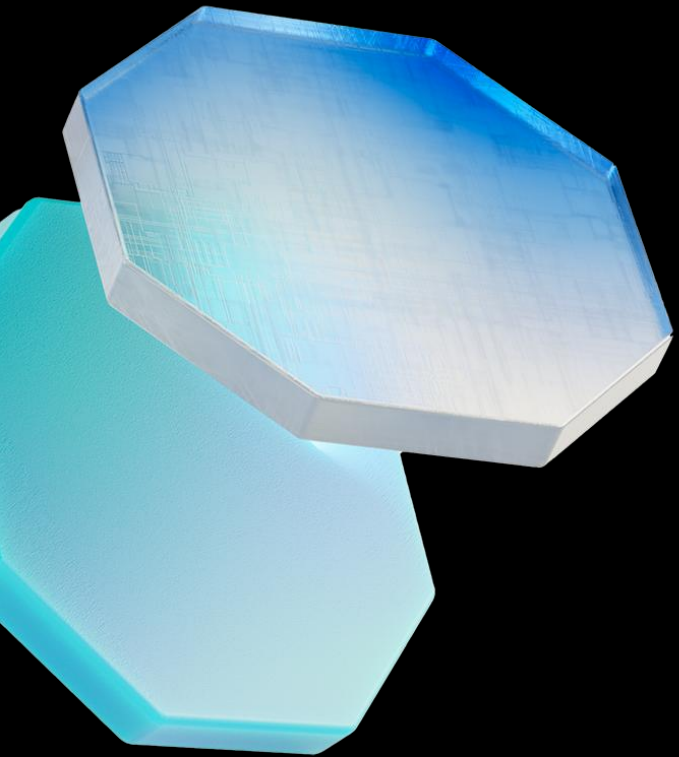
Power BI Modeling MCP Server

Microsoft  microsoft.com |  23,248 installs | ★★★★★ (5) | Free

The Power BI Modeling MCP Server, brings Power BI semantic modeling capabilities to your agents.

Install

[Trouble Installing?](#)



Demo

*Enhancing coding
agents for Power BI*

Demo 1: Power BI MCP

EXTENSIONS: MARKETPLACE

Power BI MCP

 **Power BI Modeling ...** 6ms
The Power BI Modeling MCP S...
Microsoft

 **MCP Engine - Po...** 271 ★ 5
Build, explore, and manage Po...
Maxim Anatsko

 **Power BI Translation ...** 48
Auto-configures Claude Code ...
Jason Nicolini

 **Fabric & Power BI To...** 57
Auto-configures MCP servers f...
Jason Nicolini

 **PowerPlat UPDATE M...** 15
Search and retrieve Power Plat...
yamapan

 **Power Query Lint** 19
Visual Studio Code Extension f...
Client First Technologies

 **MCP Cursor FlowiseAI** 81
Visual Flowise AI manage...
BigBrother-cursor

Extension: Power BI Modeling MCP Server X

SKILL.md ...semantic-model- ✨



Power BI Modeling MCP Server

Microsoft  45,598 | ★★★★★ (7)

The Power BI Modeling MCP Server, brings Power BI semantic modeling capabilities to your agents.

Disable | Uninstall | [Switch to Pre-Release Version](#)

☒ Auto Update

DETAILS

FEATURES

CHANGELOG

Power BI Modeling MCP Server

The **Power BI Modeling MCP Server** brings Power BI semantic modeling capabilities to your AI agents through a **local MCP server**. This allows developers and AI applications to interact with Power BI models in entirely new ways, from using natural language to execute modeling changes to autonomous AI agentic development workflows.

Power BI Desktop

Model_01 Semantic Model

CLAUDE CODE CHAT + ✖ | [] ✖

← CONNECTING TO SEMANTIC MODEL IN POWER BI []

Perfect! All 11 broken measures have been successfully fixed:

Fixed Measures:

- ✓ **Average Sale Price** - Now uses `Sales[UnitPrice]`
- ✓ **Total Revenue** - Added missing closing parenthesis
- ✓ **Customer Count** - Changed table from "Customers" to "Customer"
- ✓ **Profit Ratio** - Now references existing `[Margin]` measure
- ✓ **Sales Growth** - Fixed to use `Date` table with proper time intelligence
- ✓ **Items Per Order** - Now counts distinct invoices from Sales table
- ✓ **Tax Percentage** - Fixed table reference from "Sale" to "Sales"
- ✓ **Average Discount** - Recalculated using available columns
- ✓ **YTD Sales** - Changed from "Calendar" to "Date" table
- ✓ **Running Total** - Replaced RUNNINGSUM with proper CALCULATE formula
- ✓ **Invalid Division** - Now uses DIVIDE function to handle division by zero

All measures are now in "Ready" state with no errors. The changes have been automatically saved to your semantic model.

↻ ↶ 👍 🗨

Add Context...

Describe what to build next

🖥

📄

Auto

🔧

Demo 1: Power BI MCP – cont.

Thanks to Rui 🙌

 FabricDatabaseSchema.csv

 requirements.md

↓ requirements.md > abc # Business Requirements 📁

```
1
2  # Business Requirements 📁
3
4  | Requirement ID | Description | User Story |
  | Expected Behavior |
5  |-----|-----|-----|
  |-----|
6  | **SALES-001** | Total Sales Performance Overview |
  As a **Sales Manager**, I want to see the total sales
  amount across all channels (Internet + Reseller) so
```

CHAT

CONNECTING TO SEMANTIC MODEL IN P...

Completed

- Rebuilt a new semantic model from your updated requirements and schema as **AdventureWorks Sales v2**.
- Published it to the same workspace: **Mastering AI in Power BI**.
- Connected successfully to the published model via XMLA
(`powerbi://api.powerbi.com/v1.0/myorg/Mastering%20AI%20in%20Power%20BI`).
- Verified model objects exist: 6 tables and 4 required measures (`Total Sales Amount`, `Sales YoY Growth %`, `Sales MoM Growth %`, `Product Sales Rank`).

Fabric DB connectivity status

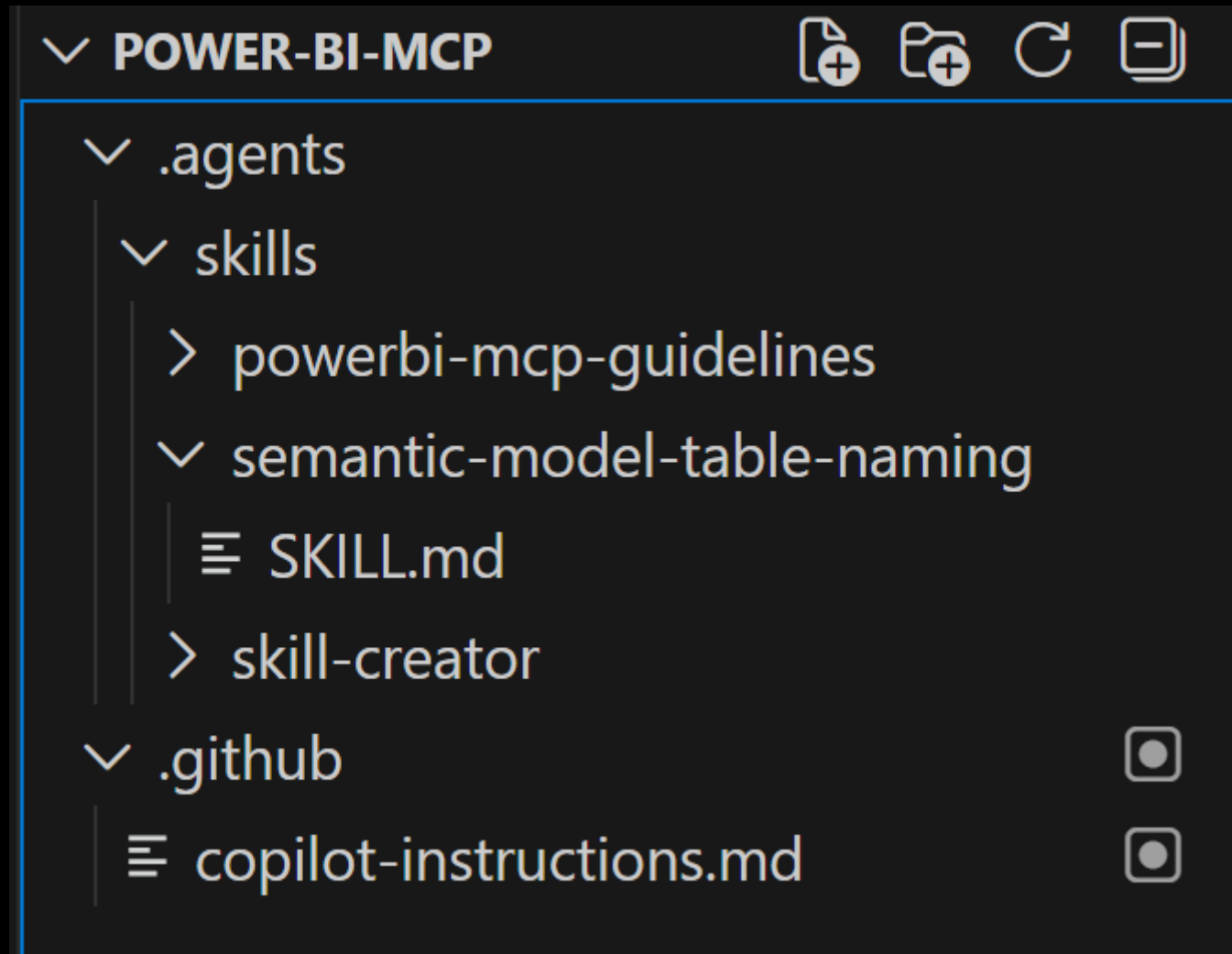
- Yes, this model is wired to your Fabric SQL endpoint.
- Refresh currently fails due to missing datasource credentials in Fabric (`DMTS_DatasourceHasNoCredentialError`), so data returns as null until credentials are set.

Add Context...

Describe what to build next

Auto

Demo 2: Agent skills + instructions + prompt



GitHub Coding Agents

